TALLER DE CODIFICACIÓN · SESIÓN 6

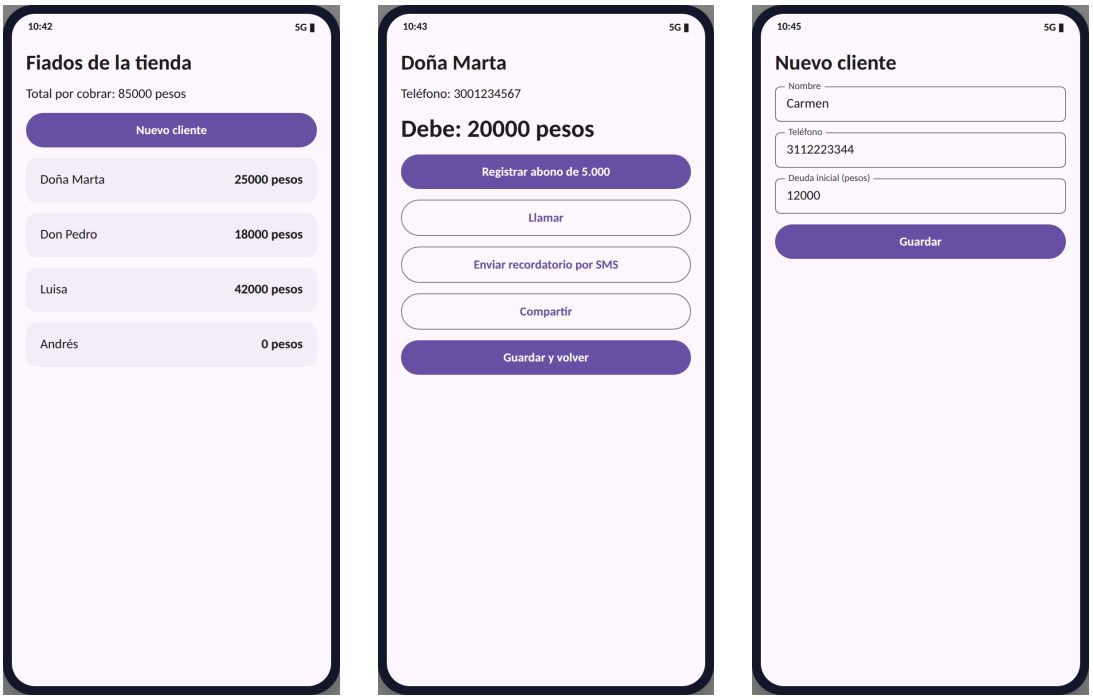
FiadoApp: Activities, Intents y ciclo de vida

Varias pantallas que se hablan: cómo una Activity abre otra, le pasa datos, recibe una respuesta y usa las apps del celular.

Asignatura	Diseño de Aplicaciones para Dispositivos Móviles	**Programa**	Ingeniería de Software
Docente	Ing. Kevin Torrez	**Duración**	80 minutos en 4 partes, más un reto extra

En la sesión 3 escribiste tu primer programa sobre el fiado de la tienda y en la sesión 5 trabajaste con una sola Activity. Hoy vas a construir **FiadoApp**, una app para que la tienda del barrio lleve sus fiados, con varias pantallas: una lista de clientes, el detalle de cada uno y un formulario. Las pantallas se abren con Intents explícitos, se pasan datos, devuelven respuestas y usan las apps del celular con Intents implícitos.

Lo que vas a construir



La lista (partes 1 y 4) *El detalle (partes 2, 3 y 4)* *Nuevo cliente (reto extra)*

- **Parte 1:** el proyecto, la lista de clientes y el total por cobrar.
- **Parte 2:** la pantalla de detalle, abierta con un Intent explícito que lleva los datos del cliente. Observas el ciclo de vida de las dos Activities en Logcat.
- **Parte 3:** tres Intents implícitos: llamar, enviar un SMS y compartir.
- **Parte 4:** el detalle devuelve la nueva deuda y la lista se actualiza.
- **Reto extra:** un formulario que devuelve un cliente nuevo.

¿Qué es? Una **Activity** es una pantalla. Cada vez que una abre otra, Android las apila: la nueva queda encima y el botón Atrás la quita de la pila. Un **Intent** es el mensaje con el que se pide abrir una Activity (explícito) o hacer una acción con cualquier app (implícito). Como cada Activity tiene su propio ciclo de vida, cuando una abre otra se ejecutan los métodos de las dos, en un orden que hoy vas a comprobar.

Antes de empezar

- Android Studio con el emulador creado. Necesitas lo mismo que en el laboratorio de la sesión 5.
- Para pegar código: Ctrl+A en el archivo, Ctrl+V y Ctrl+S. Los avisos en amarillo (por ejemplo, una importación sin usar) no impiden ejecutar; los errores en rojo sí.
- Logcat: ícono del gato en la barra lateral izquierda. Hoy filtrarás con **tag:CICLO**, que muestra los mensajes de las dos pantallas.
- Trabaja en equipo, pero cada integrante escribe el código en su computador.

Parte 1: El proyecto y la lista de clientes (15 min)

Ruta: New Project → Empty Activity → Name: FiadoApp → Package: com.example.fiadoapp → Kotlin DSL → Finish

1. Crea el proyecto **FiadoApp** con la plantilla Empty Activity. El paquete debe ser **com.example.fiadoapp**. Espera a Gradle.
2. Haz clic derecho en el paquete com.example.fiadoapp → New → Kotlin Class/File → escribe **Cliente**, elige **File** y pega:

Cliente.kt

```
package com.example.fiadoapp

/** Un cliente de la tienda y lo que debe. */
data class Cliente(
    val id: Int,
    val nombre: String,
    val telefono: String,
    val deuda: Int
)
```

3. Crea de la misma forma el archivo **ClientesViewModel** y pega:

ClientesViewModel.kt

```
package com.example.fiadoapp

import androidx.compose.runtime.mutableStateListOf
import androidx.lifecycle.ViewModel

/** Guarda la lista de clientes. Sobrevive a que el celular gire. */
class ClientesViewModel : ViewModel() {

    val clientes = mutableStateListOf(
        Cliente(1, "Doña Marta", "3001234567", 25000),
        Cliente(2, "Don Pedro", "3109876543", 18000),
        Cliente(3, "Luisa", "3205551234", 42000),
        Cliente(4, "Andrés", "3157778899", 0)
    )

    val totalPorCobrar: Int
        get() = clientes.sumOf { it.deuda }

    fun actualizarDeuda(id: Int, nuevaDeuda: Int) {
        val posicion = clientes.indexOfFirst { it.id == id }
        if (posicion >= 0) clientes[posicion] = clientes[posicion].copy(deuda = nuevaDeuda)
    }

    fun agregar(nombre: String, telefono: String, deuda: Int) {
        val nuevoId = (clientes.maxOfOrNull { it.id } ?: 0) + 1
        clientes.add(Cliente(nuevoId, nombre, telefono, deuda))
    }
}
```

4. Abre **MainActivity.kt**, selecciona todo y reemplázalo por este código. Trae ya las importaciones que necesitarás en las partes siguientes: las que aparezcan en gris son normales.

MainActivity.kt (parte 1)

```
package com.example.fiadoapp

import android.app.Activity
import android.content.Intent
import android.os.Bundle
import android.util.Log
import androidx.activity.ComponentActivity
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.activity.result.contract.ActivityResultContracts
import androidx.activity.viewModels
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material3.Button
import androidx.compose.material3.Card
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.font.FontWeight
```

```
import androidx.compose.ui.unit.dp
import com.example.fiadoapp.ui.theme.FiadoAppTheme

private const val TAG = "CICLO_MAIN"

class MainActivity : ComponentActivity() {

    private val vm: ClientesViewModel by viewModels()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d(TAG, "onCreate")
        enableEdgeToEdge()
        setContent {
            FiadoAppTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    PantallaClientes(vm, Modifier.padding(innerPadding))
                }
            }
        }
    }

    override fun onStart() { super.onStart(); Log.d(TAG, "onStart") }
    override fun onResume() { super.onResume(); Log.d(TAG, "onResume") }
    override fun onPause() { super.onPause(); Log.d(TAG, "onPause") }
    override fun onStop() { super.onStop(); Log.d(TAG, "onStop") }
    override fun onRestart() { super.onRestart(); Log.d(TAG, "onRestart") }
    override fun onDestroy() { super.onDestroy(); Log.d(TAG, "onDestroy") }
}

@Composable
fun PantallaClientes(vm: ClientesViewModel, modifier: Modifier = Modifier) {
    Column(modifier = modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(12.dp)) {
        Text(
            "Fiados de la tienda",
            style = MaterialTheme.typography.headlineSmall,
            fontWeight = FontWeight.Bold
        )
        Text("Total por cobrar: ${vm.totalPorCobrar} pesos")
        LazyColumn(verticalArrangement = Arrangement.spacedBy(8.dp)) {
            items(vm.clientes) { cliente ->
                Card(
                    modifier = Modifier.fillMaxWidth().clickable {
                        Log.d(TAG, "Tocaste a ${cliente.nombre}")
                    }
                ) {
                    Row(modifier = Modifier.padding(16.dp)) {
                        Text(cliente.nombre, modifier = Modifier.weight(1f))
                        Text("${cliente.deuda} pesos", fontWeight = FontWeight.Bold)
                    }
                }
            }
        }
    }
}
```

5. Presiona **Run ►**.

> **Lo que debes ver:** la lista con cuatro clientes y el total por cobrar: 85000 pesos. Al tocar un cliente todavía no pasa nada en pantalla, pero en Logcat (tag:CICLO) aparece «Tocaste a …».

Qué hace

- **Cliente** es una data class: solo guarda datos.
- **ClientesViewModel** guarda la lista en un **mutableStateListOf**: cuando cambia un cliente, Compose redibuja la lista. Como es un ViewModel, sobrevive al giro del celular.
- **MainActivity** escribe en Logcat cada método del ciclo de vida, como en la sesión 5, con la etiqueta CICLO_MAIN.
- **LazyColumn** con **items** dibuja una tarjeta por cliente; **clickable** reacciona al toque.

Parte 2: La pantalla de detalle y un Intent con datos (20 min)

¿Qué es? Un **Intent explícito** nombra la clase exacta que quieres abrir. Con **putExtra** le cuelgas datos (textos, números, booleanos) y la otra Activity los lee con **getStringExtra** o **getIntExtra**. Las claves van en constantes para no equivocarse al escribirlas.

Ruta: [Clic derecho en el paquete] → New → Activity → Empty Activity → Activity Name: DetalleActivity → Finish

1. Crea la Activity con el asistente, como dice la ruta. Deja marcada la casilla **Generate a Layout File** apagada si aparece, y desmarca **Launcher Activity**. Android Studio crea el archivo y, además, declara la Activity en el Manifest.
2. Abre **AndroidManifest.xml** y comprueba que apareció esta declaración dentro de **<application>**:

AndroidManifest.xml

```
<activity
    android:name=".DetalleActivity"
    android:exported="false" />
```

Si creaste el archivo con New → Kotlin Class/File en vez del asistente, agrega tú esas tres líneas al Manifest. Sin ellas, la app se cierra al abrir el detalle.

3. Reemplaza todo el contenido de **DetalleActivity.kt** por este código:

DetalleActivity.kt (parte 2)

```
package com.example.fiadoapp

import android.content.Intent
import android.net.Uri
import android.os.Bundle
import android.util.Log
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
```

```
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.fiadoapp.ui.theme.FiadoAppTheme

private const val TAG = "CICLO_DETALLE"

class DetalleActivity : ComponentActivity() {

    companion object {
        const val EXTRA_ID = "id"
        const val EXTRA_NOMBRE = "nombre"
        const val EXTRA_TELEFONO = "telefono"
        const val EXTRA_DEUDA = "deuda"
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d(TAG, "onCreate")
        enableEdgeToEdge()

        // Los datos llegan en el Intent que abrió esta pantalla.
        val id = intent.getIntExtra(EXTRA_ID, -1)
        val nombre = intent.getStringExtra(EXTRA_NOMBRE) ?: "Sin nombre"
        val telefono = intent.getStringExtra(EXTRA_TELEFONO) ?: ""
        val deudaInicial = intent.getIntExtra(EXTRA_DEUDA, 0)

        setContent {
            FiadoAppTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    PantallaDetalle(
                        nombre = nombre,
                        telefono = telefono,
                        deudaInicial = deudaInicial,
                        onGuardar = { nuevaDeuda ->
                            Log.d(TAG, "Nueva deuda: $nuevaDeuda (todavía no se devuelve)")
                            finish()
                        },
                        modifier = Modifier.padding(innerPadding)
                    )
                }
            }
        }
    }

    override fun onStart() { super.onStart(); Log.d(TAG, "onStart") }
    override fun onResume() { super.onResume(); Log.d(TAG, "onResume") }
    override fun onPause() { super.onPause(); Log.d(TAG, "onPause") }
    override fun onStop() { super.onStop(); Log.d(TAG, "onStop") }
    override fun onRestart() { super.onRestart(); Log.d(TAG, "onRestart") }
    override fun onDestroy() { super.onDestroy(); Log.d(TAG, "onDestroy") }
}
@Composable
```

```
fun PantallaDetalle(
    nombre: String,
    telefono: String,
    deudaInicial: Int,
    onGuardar: (Int) -> Unit,
    modifier: Modifier = Modifier
) {
    val context = LocalContext.current
    var deuda by rememberSaveable { mutableStateOf(deudaInicial) }

    Column(modifier = modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(12.dp)) {
        Text(nombre, style = MaterialTheme.typography.headlineSmall, fontWeight = FontWeight.Bold)
        Text("Teléfono: $telefono")
        Text("Debe: $deuda pesos", fontSize = 28.sp, fontWeight = FontWeight.Bold)

        Button(
            onClick = { deuda = (deuda - 5000).coerceAtLeast(0) },
            modifier = Modifier.fillMaxWidth()
        ) { Text("Registrar abono de 5.000") }

        Button(onClick = { onGuardar(deuda) }, modifier = Modifier.fillMaxWidth()) {
            Text("Guardar y volver")
        }
    }
}
```

4. En **MainActivity.kt**, busca el bloque **clickable** (Ctrl+F) y reemplázalo por este, que arma el Intent y abre el detalle:

MainActivity.kt: el bloque clickable, segunda versión

```
modifier = Modifier.fillMaxWidth().clickable {
    val intent = Intent(context, DetalleActivity::class.java)
    intent.putExtra(DetalleActivity.EXTRA_ID, cliente.id)
    intent.putExtra(DetalleActivity.EXTRA_NOMBRE, cliente.nombre)
    intent.putExtra(DetalleActivity.EXTRA_TELEFONO, cliente.telefor
    intent.putExtra(DetalleActivity.EXTRA_DEUDA, cliente.deuda)
    context.startActivity(intent)
}
```

5. Presiona **Run ►**, toca un cliente, registra un abono y vuelve con el botón Atrás.

> **Lo que debes ver:** el detalle con el nombre, el teléfono y la deuda del cliente que tocaste. Al volver, la lista sigue igual: la nueva deuda todavía no viaja de regreso. Eso lo arreglas en la parte 4.

Experimento: dos Activities en Logcat

6. Limpia Logcat (ícono de la papelera), filtra con **tag:CICLO** y toca un cliente. Verás que las dos pantallas se turnan:

Al abrir el detalle:

```
CICLO_MAIN     onPause
CICLO_DETALLE  onCreate
CICLO_DETALLE  onStart
CICLO_DETALLE  onResume
CICLO_MAIN     onStop
```

7. Ahora presiona Atrás:

Al volver a la lista:

```
CICLO_DETALLE  onPause
CICLO_MAIN     onRestart
CICLO_MAIN     onStart
CICLO_MAIN     onResume
CICLO_DETALLE  onStop
CICLO_DETALLE  onDestroy
```

Fíjate en el orden: la lista se pausa antes de que el detalle se cree, pero solo se detiene (onStop) cuando el detalle ya está en primer plano. Y al volver, la lista reaparece con onRestart, no con onCreate: nunca se destruyó, estaba debajo en la pila. El detalle sí se destruye, porque Atrás lo saca de la pila.

Parte 3: Intents implícitos: llamar, escribir y compartir (20 min)

¿Qué es? Un **Intent implícito** no nombra una clase: dice qué acción quieres (ACTION_DIAL, ACTION_SENDTO, ACTION_SEND) y sobre qué datos (un tel:, un smsto:, un texto). Android busca entre las apps instaladas cuál puede hacerlo. Si hay varias, muestra un selector.

1. En **DetalleActivity.kt**, busca la línea **Button(onClick = { onGuardar(deuda) }** y pega **antes** de ella este bloque:

DetalleActivity.kt: tres botones nuevos, antes de «Guardar y volver»

```
        // Intents implícitos: Android busca qué app puede hacer cada cosa.
        OutlinedButton(
            onClick = {
                val intent = Intent(Intent.ACTION_DIAL, Uri.parse("tel:$telefono"))
                context.startActivity(intent)
            },
            modifier = Modifier.fillMaxWidth()
        ) { Text("Llamar") }

        OutlinedButton(
            onClick = {
                val intent = Intent(Intent.ACTION_SENDTO, Uri.parse("smsto:$telefono"))
                val mensaje = "Hola $nombre, te recordamos que tu fiado es de $deuda pe
                intent.putExtra("sms_body", mensaje)
                context.startActivity(intent)
            },
            modifier = Modifier.fillMaxWidth()
        ) { Text("Enviar recordatorio por SMS") }

        OutlinedButton(
            onClick = {
                val intent = Intent(Intent.ACTION_SEND)
                intent.type = "text/plain"
                intent.putExtra(Intent.EXTRA_TEXT, "Fiado de $nombre: $deuda pesos")
                context.startActivity(Intent.createChooser(intent, "Compartir con"))
            },
            modifier = Modifier.fillMaxWidth()
        ) { Text("Compartir") }
```

2. Presiona **Run ►**, abre un cliente y prueba los tres botones. Vuelve a tu app con Atrás después de cada uno.

> **Lo que debes ver: Llamar** abre el teléfono con el número escrito, sin llamar. **Enviar recordatorio** abre Mensajes con el texto listo. **Compartir** muestra la ventana «Compartir con».

Qué hace

- **ACTION_DIAL** con **tel:** abre el marcador; la persona decide si llama. Por eso no necesita permiso. ACTION_CALL llamaría directo y sí lo necesita.
- **ACTION_SENDTO** con **smsto:** abre la app de mensajes con el destinatario; el extra **sms_body** lleva el texto.
- **ACTION_SEND** con **type = "text/plain"** ofrece el texto a cualquier app que reciba texto; **createChooser** fuerza el selector.
- Otro muy útil: **ACTION_VIEW** con **geo:0,0?q=Tienda+La+Esquina** abre el mapa buscando ese lugar.

Parte 4: El detalle devuelve la nueva deuda (25 min)

¿Qué es? Una Activity puede **responder** a la que la abrió. La que abre usa un **launcher** (Activity Result API) en vez de startActivity; la que responde arma un Intent con los datos, llama a **setResult(RESULT_OK, intent)** y a **finish()**. El launcher recibe la respuesta y actualiza el ViewModel.

1. Reemplaza todo **DetalleActivity.kt** por su versión final. Solo cambia el bloque **onGuardar**: ahora devuelve el id y la deuda.

DetalleActivity.kt (versión final)

```
package com.example.fiadoapp

import android.content.Intent
import android.net.Uri
import android.os.Bundle
import android.util.Log
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.fiadoapp.ui.theme.FiadoAppTheme

private const val TAG = "CICLO_DETALLE"

class DetalleActivity : ComponentActivity() {

    companion object {
        const val EXTRA_ID = "id"
        const val EXTRA_NOMBRE = "nombre"
        const val EXTRA_TELEFONO = "telefono"
        const val EXTRA_DEUDA = "deuda"
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d(TAG, "onCreate")
        enableEdgeToEdge()

        // Los datos llegan en el Intent que abrió esta pantalla.
        val id = intent.getIntExtra(EXTRA_ID, -1)
        val nombre = intent.getStringExtra(EXTRA_NOMBRE) ?: "Sin nombre"
```

```
        val telefono = intent.getStringExtra(EXTRA_TELEFONO) ?: ""
        val deudaInicial = intent.getIntExtra(EXTRA_DEUDA, 0)

        setContent {
            FiadoAppTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    PantallaDetalle(
                        nombre = nombre,
                        telefono = telefono,
                        deudaInicial = deudaInicial,
                        onGuardar = { nuevaDeuda ->
                            // La respuesta viaja de vuelta en otro Intent.
                            val respuesta = Intent()
                            respuesta.putExtra(EXTRA_ID, id)
                            respuesta.putExtra(EXTRA_DEUDA, nuevaDeuda)
                            setResult(RESULT_OK, respuesta)
                            finish()
                        },
                        modifier = Modifier.padding(innerPadding)
                    )
                }
            }
        }
    }

    override fun onStart() { super.onStart(); Log.d(TAG, "onStart") }
    override fun onResume() { super.onResume(); Log.d(TAG, "onResume") }
    override fun onPause() { super.onPause(); Log.d(TAG, "onPause") }
    override fun onStop() { super.onStop(); Log.d(TAG, "onStop") }
    override fun onRestart() { super.onRestart(); Log.d(TAG, "onRestart") }
    override fun onDestroy() { super.onDestroy(); Log.d(TAG, "onDestroy") }
}

@Composable
fun PantallaDetalle(
    nombre: String,
    telefono: String,
    deudaInicial: Int,
    onGuardar: (Int) -> Unit,
    modifier: Modifier = Modifier
) {
    val context = LocalContext.current
    var deuda by rememberSaveable { mutableStateOf(deudaInicial) }

    Column(modifier = modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(12.dp)) {
        Text(nombre, style = MaterialTheme.typography.headlineSmall, fontWeight = FontWeight.Bold)
        Text("Teléfono: $telefono")
        Text("Debe: $deuda pesos", fontSize = 28.sp, fontWeight = FontWeight.Bold)

        Button(
            onClick = { deuda = (deuda - 5000).coerceAtLeast(0) },
            modifier = Modifier.fillMaxWidth()
        ) { Text("Registrar abono de 5.000") }

        // Intents implícitos: Android busca qué app puede hacer cada cosa.
        OutlinedButton(
            onClick = {
                val intent = Intent(Intent.ACTION_DIAL, Uri.parse("tel:$telefono"))
                context.startActivity(intent)
            },
            modifier = Modifier.fillMaxWidth()
        ) { Text("Llamar") }

        OutlinedButton(
```

```
            onClick = {
                val intent = Intent(Intent.ACTION_SENDTO, Uri.parse("smsto:$telefono"))
                val mensaje = "Hola $nombre, te recordamos que tu fiado es de $deuda pesos."
                intent.putExtra("sms_body", mensaje)
                context.startActivity(intent)
            },
            modifier = Modifier.fillMaxWidth()
        ) { Text("Enviar recordatorio por SMS") }

        OutlinedButton(
            onClick = {
                val intent = Intent(Intent.ACTION_SEND)
                intent.type = "text/plain"
                intent.putExtra(Intent.EXTRA_TEXT, "Fiado de $nombre: $deuda pesos")
                context.startActivity(Intent.createChooser(intent, "Compartir con"))
            },
            modifier = Modifier.fillMaxWidth()
        ) { Text("Compartir") }

        Button(onClick = { onGuardar(deuda) }, modifier = Modifier.fillMaxWidth()) {
            Text("Guardar y volver")
        }
    }
}
```

2. En **MainActivity.kt**, reemplaza la función **PantallaClientes** completa (desde @Composable hasta su última llave) por esta:

MainActivity.kt: la función PantallaClientes, con el launcher

```
@Composable
fun PantallaClientes(vm: ClientesViewModel, modifier: Modifier = Modifier) {
    val context = LocalContext.current

    // Abre DetalleActivity y recibe la nueva deuda cuando esa pantalla termina.
    val abrirDetalle = rememberLauncherForActivityResult(
        ActivityResultContracts.StartActivityForResult()
    ) { resultado ->
        val datos = resultado.data
        if (resultado.resultCode == Activity.RESULT_OK && datos != null) {
            val id = datos.getIntExtra(DetalleActivity.EXTRA_ID, -1)
            val deuda = datos.getIntExtra(DetalleActivity.EXTRA_DEUDA, 0)
            vm.actualizarDeuda(id, deuda)
        }
    }

    Column(modifier = modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(12.dp)) {
        Text(
            "Fiados de la tienda",
            style = MaterialTheme.typography.headlineSmall,
            fontWeight = FontWeight.Bold
        )
        Text("Total por cobrar: ${vm.totalPorCobrar} pesos")
        LazyColumn(verticalArrangement = Arrangement.spacedBy(8.dp)) {
            items(vm.clientes) { cliente ->
                Card(
                    modifier = Modifier.fillMaxWidth().clickable {
                        val intent = Intent(context, DetalleActivity::class.java)
                        intent.putExtra(DetalleActivity.EXTRA_ID, cliente.id)
                        intent.putExtra(DetalleActivity.EXTRA_NOMBRE, cliente.nombre)
                        intent.putExtra(DetalleActivity.EXTRA_TELEFONO, cliente.telefono)
                        intent.putExtra(DetalleActivity.EXTRA_DEUDA, cliente.deuda)
                        abrirDetalle.launch(intent)
                    }
```

```
                ) {
                    Row(modifier = Modifier.padding(16.dp)) {
                        Text(cliente.nombre, modifier = Modifier.weight(1f))
                        Text("${cliente.deuda} pesos", fontWeight = FontWeight.Bold)
                    }
                }
            }
        }
    }
}
```

3. Presiona **Run ►**. Abre un cliente, registra dos abonos y toca **Guardar y volver**.

> **Lo que debes ver:** la lista con la deuda actualizada y el total por cobrar recalculado. Gira el celular: la lista conserva los cambios porque viven en el ViewModel.

Qué hace

- **rememberLauncherForActivityResult** con **StartActivityForResult** crea un launcher: abre la Activity y espera su respuesta.
- En la respuesta, **resultCode** dice si la pantalla terminó bien (RESULT_OK) o se cerró con Atrás (RESULT_CANCELED). Los datos vienen en **resultado.data**.
- La lista no lee el detalle directamente: el detalle responde y el ViewModel actualiza. Cada pantalla sigue teniendo un solo trabajo.

Reto extra: un formulario que devuelve un cliente nuevo

1. Crea con el asistente la Activity **NuevoClienteActivity** (New → Activity → Empty Activity) y reemplaza su contenido por:

NuevoClienteActivity.kt

```
package com.example.fiadoapp

import android.content.Intent
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material3.Button
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
```

```
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.unit.dp
import com.example.fiadoapp.ui.theme.FiadoAppTheme

/** Reto extra: una pantalla que devuelve un cliente nuevo a la lista. */
class NuevoClienteActivity : ComponentActivity() {

    companion object {
        const val EXTRA_NOMBRE = "nombre"
        const val EXTRA_TELEFONO = "telefono"
        const val EXTRA_DEUDA = "deuda"
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            FiadoAppTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    FormularioCliente(
                        onGuardar = { nombre, telefono, deuda ->
                            val respuesta = Intent()
                            respuesta.putExtra(EXTRA_NOMBRE, nombre)
                            respuesta.putExtra(EXTRA_TELEFONO, telefono)
                            respuesta.putExtra(EXTRA_DEUDA, deuda)
                            setResult(RESULT_OK, respuesta)
                            finish()
                        },
                        modifier = Modifier.padding(innerPadding)
                    )
                }
            }
        }
    }
}

@Composable
fun FormularioCliente(onGuardar: (String, String, Int) -> Unit, modifier: Modifier = Modifier) {
    var nombre by rememberSaveable { mutableStateOf("") }
    var telefono by rememberSaveable { mutableStateOf("") }
    var deuda by rememberSaveable { mutableStateOf("") }

    Column(modifier = modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(12.dp)) {
        Text(
            "Nuevo cliente",
            style = MaterialTheme.typography.headlineSmall,
            fontWeight = FontWeight.Bold
        )
        OutlinedTextField(
            value = nombre,
            onValueChange = { nombre = it },
            label = { Text("Nombre") },
            singleLine = true,
            modifier = Modifier.fillMaxWidth()
        )
        OutlinedTextField(
            value = telefono,
            onValueChange = { telefono = it.filter { c -> c.isDigit() } },
            label = { Text("Teléfono") },
            singleLine = true,
            keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Phone),
            modifier = Modifier.fillMaxWidth()
        )
        OutlinedTextField(
```

```
            value = deuda,
            onValueChange = { deuda = it.filter { c -> c.isDigit() } },
            label = { Text("Deuda inicial (pesos)") },
            singleLine = true,
            keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
            modifier = Modifier.fillMaxWidth()
        )
        Button(
            onClick = { onGuardar(nombre.trim(), telefono, deuda.toIntOrNull() ?: 0) },
            enabled = nombre.isNotBlank(),
            modifier = Modifier.fillMaxWidth()
        ) { Text("Guardar") }
    }
}
```

2. Comprueba en el Manifest que quedó declarada, igual que DetalleActivity.
3. En **MainActivity.kt**, reemplaza de nuevo la función **PantallaClientes** por esta versión final, que agrega el botón y el segundo launcher:

MainActivity.kt: la función PantallaClientes, versión final

```
@Composable
fun PantallaClientes(vm: ClientesViewModel, modifier: Modifier = Modifier) {
    val context = LocalContext.current

    // Abre DetalleActivity y recibe la nueva deuda cuando esa pantalla termina.
    val abrirDetalle = rememberLauncherForActivityResult(
        ActivityResultContracts.StartActivityForResult()
    ) { resultado ->
        val datos = resultado.data
        if (resultado.resultCode == Activity.RESULT_OK && datos != null) {
            val id = datos.getIntExtra(DetalleActivity.EXTRA_ID, -1)
            val deuda = datos.getIntExtra(DetalleActivity.EXTRA_DEUDA, 0)
            vm.actualizarDeuda(id, deuda)
        }
    }

    // Reto extra: abre NuevoClienteActivity y recibe el cliente nuevo.
    val abrirNuevo = rememberLauncherForActivityResult(
        ActivityResultContracts.StartActivityForResult()
    ) { resultado ->
        val datos = resultado.data
        if (resultado.resultCode == Activity.RESULT_OK && datos != null) {
            vm.agregar(
                nombre = datos.getStringExtra(NuevoClienteActivity.EXTRA_NOMBRE) ?: "",
                telefono = datos.getStringExtra(NuevoClienteActivity.EXTRA_TELEFONO) ?: "",
                deuda = datos.getIntExtra(NuevoClienteActivity.EXTRA_DEUDA, 0)
            )
        }
    }

    Column(modifier = modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(12.dp)) {
        Text(
            "Fiados de la tienda",
            style = MaterialTheme.typography.headlineSmall,
            fontWeight = FontWeight.Bold
        )
        Text("Total por cobrar: ${vm.totalPorCobrar} pesos")
        Button(
            onClick = { abrirNuevo.launch(Intent(context, NuevoClienteActivity::class.java)) },
            modifier = Modifier.fillMaxWidth()
        ) { Text("Nuevo cliente") }
        LazyColumn(verticalArrangement = Arrangement.spacedBy(8.dp)) {
```

```
            items(vm.clientes) { cliente ->
                Card(
                    modifier = Modifier.fillMaxWidth().clickable {
                        val intent = Intent(context, DetalleActivity::class.java)
                        intent.putExtra(DetalleActivity.EXTRA_ID, cliente.id)
                        intent.putExtra(DetalleActivity.EXTRA_NOMBRE, cliente.nombre)
                        intent.putExtra(DetalleActivity.EXTRA_TELEFONO, cliente.telefono)
                        intent.putExtra(DetalleActivity.EXTRA_DEUDA, cliente.deuda)
                        abrirDetalle.launch(intent)
                    }
                ) {
                    Row(modifier = Modifier.padding(16.dp)) {
                        Text(cliente.nombre, modifier = Modifier.weight(1f))
                        Text("${cliente.deuda} pesos", fontWeight = FontWeight.Bold)
                    }
                }
            }
        }
    }
}
```

Lo que debes ver: el botón Nuevo cliente arriba de la lista; al guardar el formulario, el cliente aparece al final de la lista y el total cambia.

Reto para su proyecto: el mapa de pantallas

Con lo de hoy, su equipo ya puede dibujar la estructura de la app del proyecto. Llenen esta tabla en la bitácora: una fila por pantalla. Va en la Entrega 1 de la sesión 7.

Pantalla (Activity)	Qué muestra	Desde dónde se abre y con qué datos	Intents implícitos que usa
Ejemplo: DetalleActivity	La deuda de un cliente y sus acciones	Desde la lista, con id, nombre, teléfono y deuda	Llamar (DIAL), SMS (SENDTO), compartir (SEND)
1.			
2.			
3.			
4.			

Marquen con una estrella la pantalla más importante: es la que conviene construir primero en el segundo corte.

Evidencia del taller y cómo se califica

- Dos capturas: la lista después de un abono y el detalle de un cliente.
- La secuencia de Logcat al abrir el detalle y al volver, escrita en la bitácora (puedes copiarla de la parte 2 si te salió igual, o anotar lo que viste si fue distinto).
- El mapa de pantallas del proyecto.
- La carpeta del proyecto comprimida (sin la carpeta build) subida al aula virtual, o el enlace de su repositorio en GitHub si ya lo usan.

Criterio	Peso	Qué se evalúa
Conocimiento	30 %	Explican la secuencia del ciclo de vida entre dos Activities y la diferencia entre Intent explícito e implícito.
Habilidad	30 %	FiadoApp funciona: abre el detalle con datos, usa los Intents implícitos y devuelve la deuda.
Acción	40 %	El reto extra o el mapa de pantallas del proyecto, con decisiones propias.
Actitud	Factor	Puntualidad, trabajo en equipo y tolerancia al error. Multiplica la nota.

Problemas comunes

Lo que ves	Causa probable	Solución
«Unresolved reference: DetalleActivity»	El archivo no existe o tiene otro nombre	Créalo con el asistente de la parte 2 y escribe el nombre exacto, con mayúsculas.
La app se cierra al tocar un cliente y Logcat dice «have you declared this activity in your AndroidManifest.xml?»	La Activity no está en el Manifest	Agrega las tres líneas de la parte 2 dentro de <application>.
El detalle muestra «Sin nombre» o deuda 0	Las claves de putExtra y getExtra no coinciden	Usa siempre las constantes EXTRA_… de DetalleActivity, nunca textos escritos a mano.
«Unresolved reference: FiadoAppTheme»	El proyecto no se llama FiadoApp	Cambia FiadoAppTheme por el nombre del tema de tu proyecto, en ui.theme → Theme.kt.
Al volver con Atrás la lista no cambia	Es lo esperado hasta la parte 4, o se usó Atrás en vez de «Guardar y volver»	Atrás devuelve RESULT_CANCELED; solo «Guardar y volver» envía la respuesta.
«Llamar» o «SMS» no hacen nada en el emulador	El emulador no tiene app para esa acción	Usa un emulador con Google APIs (los Pixel los traen) o prueba en un celular real.
Subrayados rojos justo después de pegar	Android Studio sigue indexando	Espera unos segundos; si siguen, [Menú ☰] → Build → Make Project.